Phase III: Software Design and Modelling

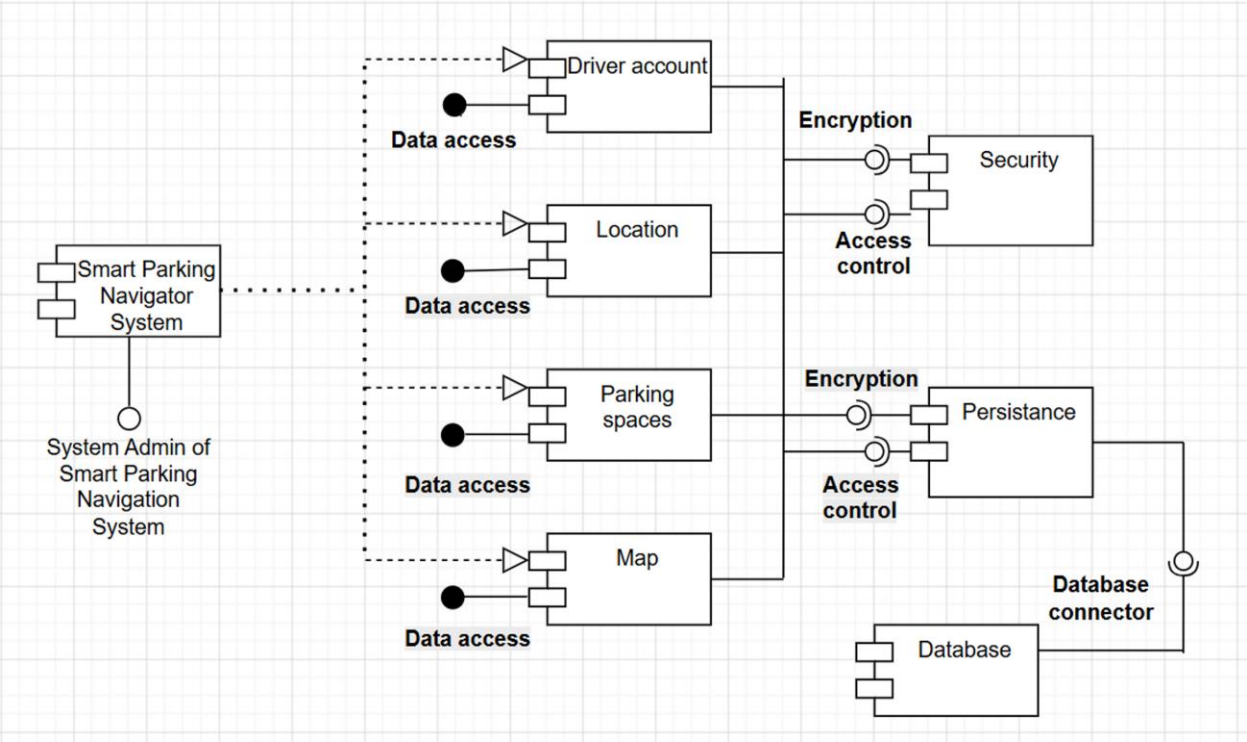
1.Software Architecture

Software Architecture

- Driver/User opens the application and either enters a location manually or allows automatic location detection (Frontend)
- Frontend captures the input and sends a request to the Backend via API
- Backend receives the request and performs input validation (checks for valid location data)
- If input is invalid, an error message is sent back to the Frontend and displayed to the user
- If valid, Backend processes the request and queries the Database for:
    a) Parking locations
    b) Availability status of parking spaces
- Backend may also integrate with an external Map API (e.g., Google Maps) to retrieve map-related data
- Backend processes and organizes the data (filters, nearest parking, availability highlighting)
- Processed data is sent back to the Frontend
- Frontend displays the results on an interactive map interface with markers indicating available and unavailable parking
- User interacts with the system by:
    a) Viewing parking details
    b) Filtering results (e.g., nearest parking)
    c) Refreshing search results
- When the user refreshes or performs a new action, the system requests updated data from the Backend
- Backend retrieves updated information from the Database to ensure real-time availability

- Updated results are displayed again on the Frontend

Component Diagrams



Left part

- Smart Parking Navigator System: Acts as the main frontend entry point of the application, intercepting driver requests and routing them to the correct core middle modules.

Components (middle part)

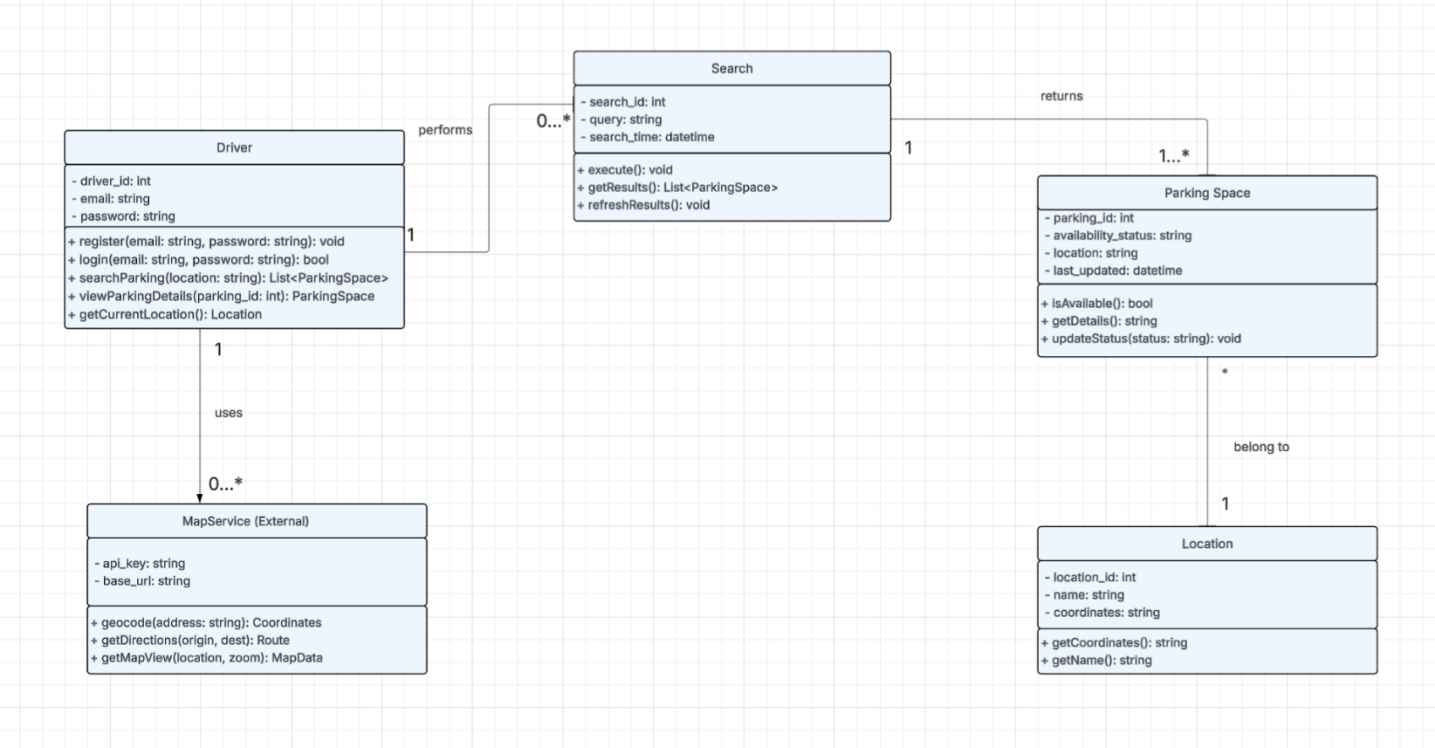
- Driver:Manages driver account registration, profile data, and login history.
- Location: Handles tracking and updating the driver's geographic coordinates.
- Parking: Processes the status updates and availability logic for parking spaces and advance space reservations for the driver.
- Map: Manages the visual rendering of nearby parking locations on the driver's interface.

Right part

- Security: Manages authorization and encrypts sensitive information to ensure secure data handling.
- Persistence: Coordinates long-term data transactions, ensuring runtime information maps correctly to physical storage.
- Database: Provides the persistent storage tier where user data, locations, and parking space records are securely held.
- Database Connector: Serves as the direct interface linking the persistence layer to the database instance.

2.Detailed Design

Class Diagrams

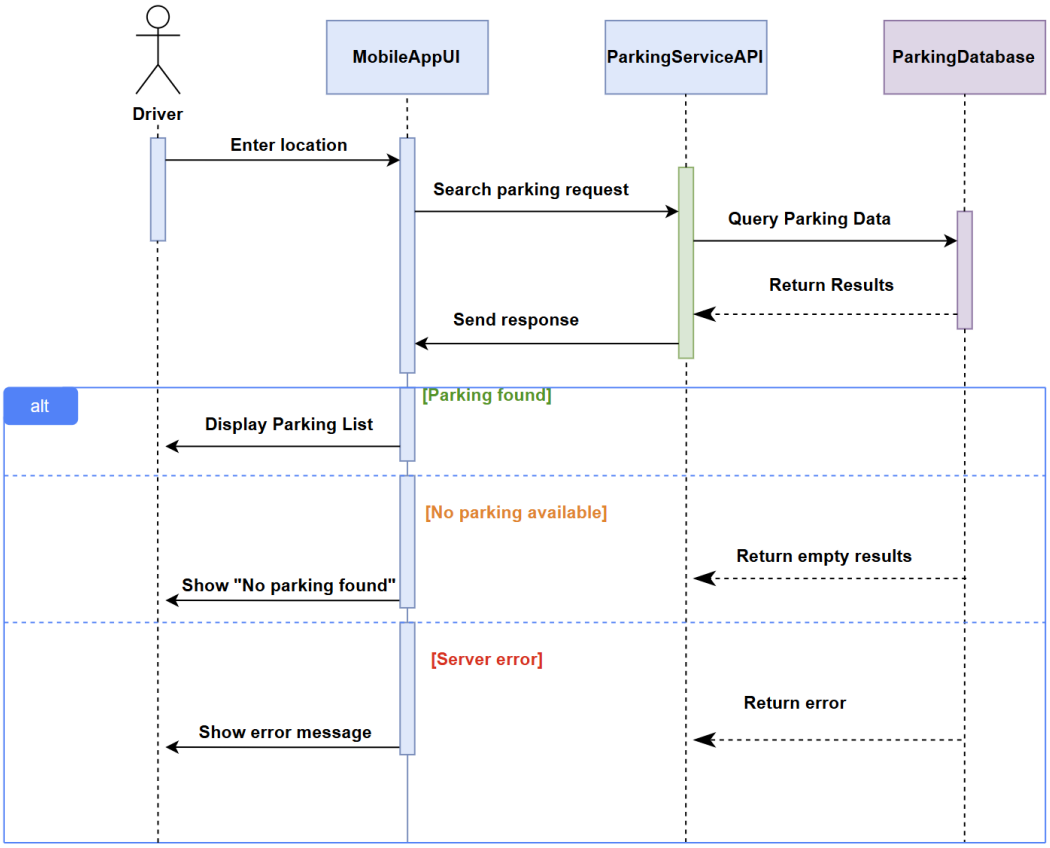


The class diagram represents the main structure of the Smart Parking Navigator system. It shows the main classes of the application, their attributes, methods, and the relationships between them. The diagram helps explain how the system is organized and how the classes interact with each other.

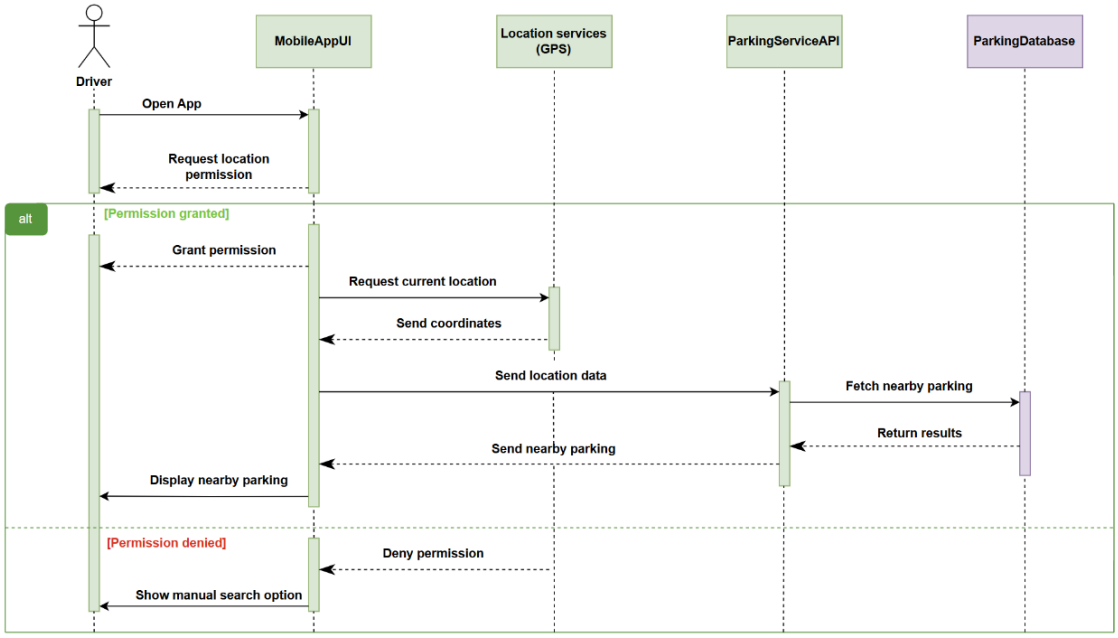
- Driver: Represents the user of the system. The driver can search for parking, view details, and use location services.
- Search: Handles the parking search process and stores search data like query and time.
- ParkingSpace: Represents each parking spot, including its availability and location.
- Location: Stores location details such as name and coordinates. One location can have multiple parking spaces.
- MapService: External service used for map display, directions, and location conversion.

Sequence Diagrams

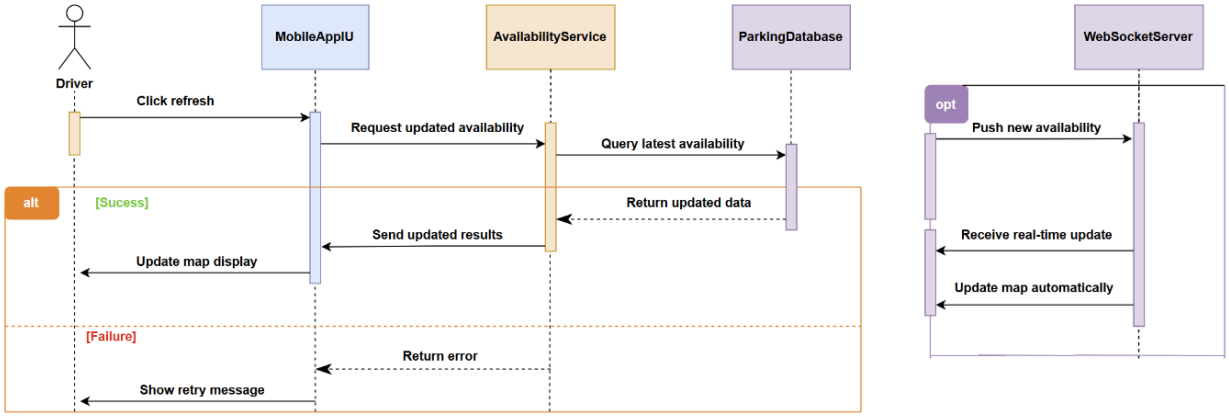
Sequence Diagram 1: Search Parking



Sequence Diagram 2: Detect Location

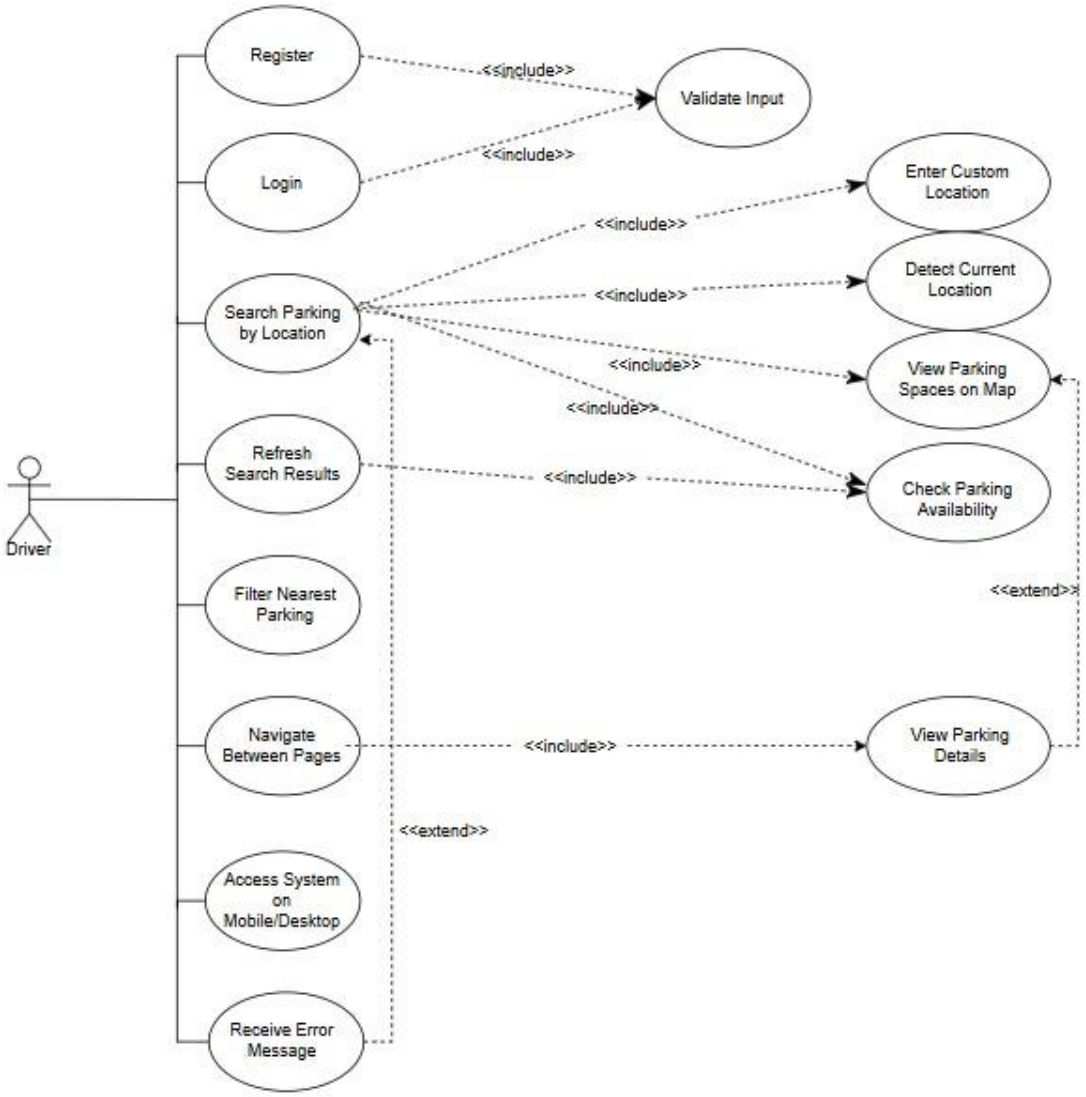


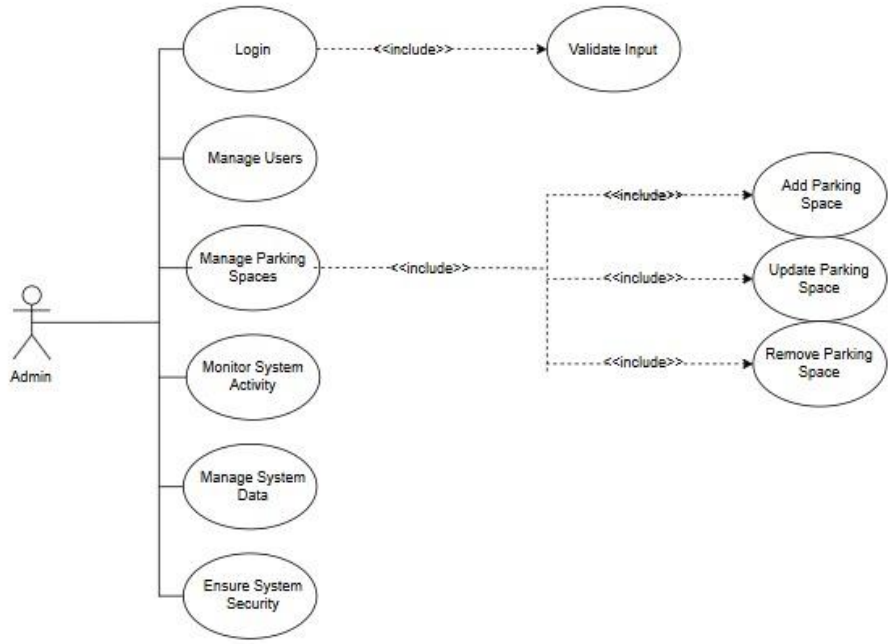
Sequence Diagram 3: Refresh / Real-Time Update



3.Modeling

Use Case Diagrams





(The Use Case diagram is 1 diagram separated into 2 screenshots)

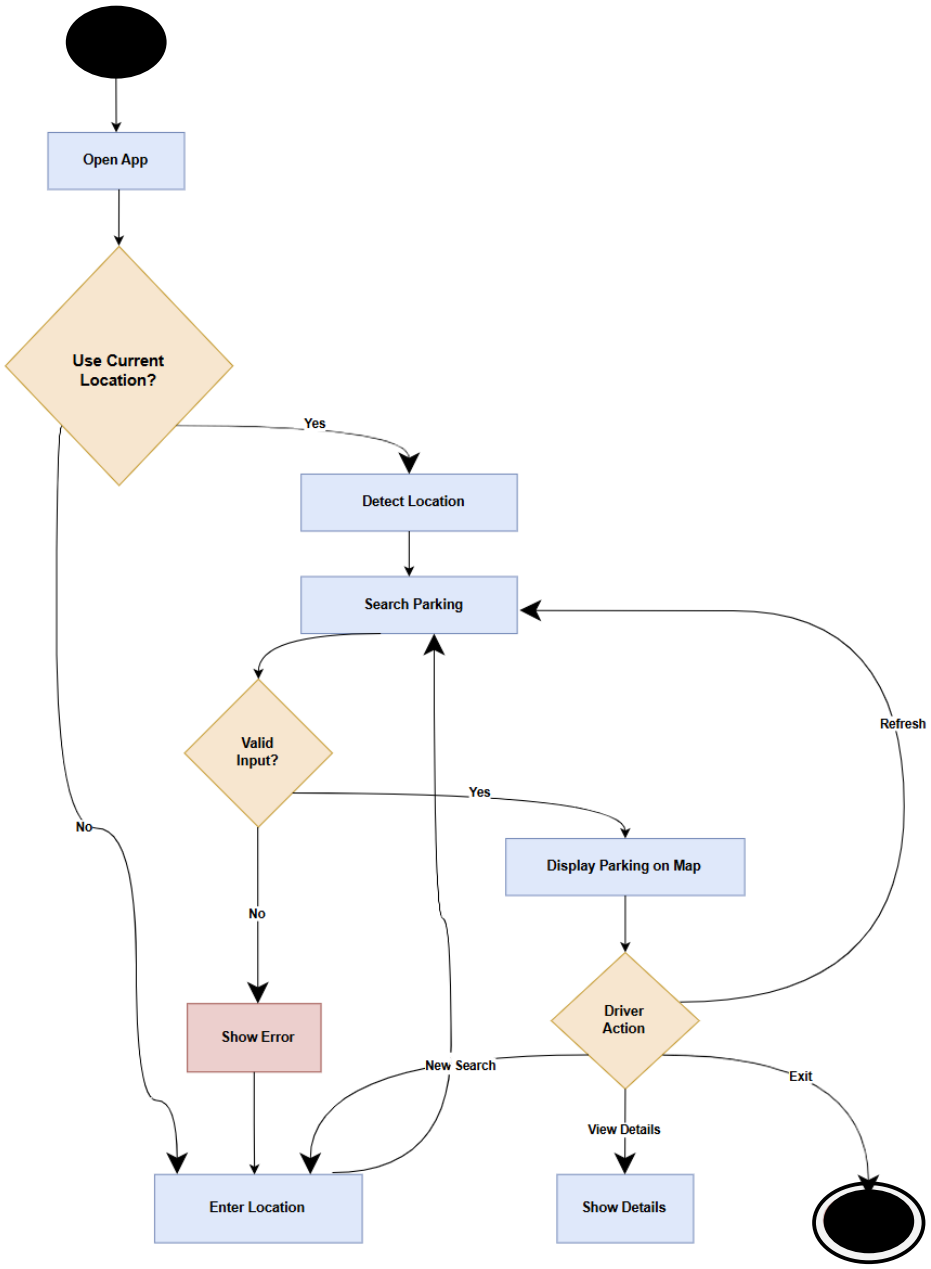
This use case diagram presents how users interact with the Smart Parking Navigator System. The system includes two main actors: the Driver and the Admin.

The Driver can perform actions such as registering, logging in and searching for parking by location. The search functionality includes entering a custom location, detecting the current location, viewing parking spaces on a map, and checking parking availability. The driver can also refresh results, filter nearby parking, navigate between pages and also view parking details. In case of invalid input or system issues, the system provides error messages as an optional behavior.

The Admin is responsible for managing the system. The admin can log in, manage users, and manage parking spaces. The management of parking spaces includes adding, updating and removing parking spaces. Additionally, the admin can monitor system activity, manage system data and ensure system security.

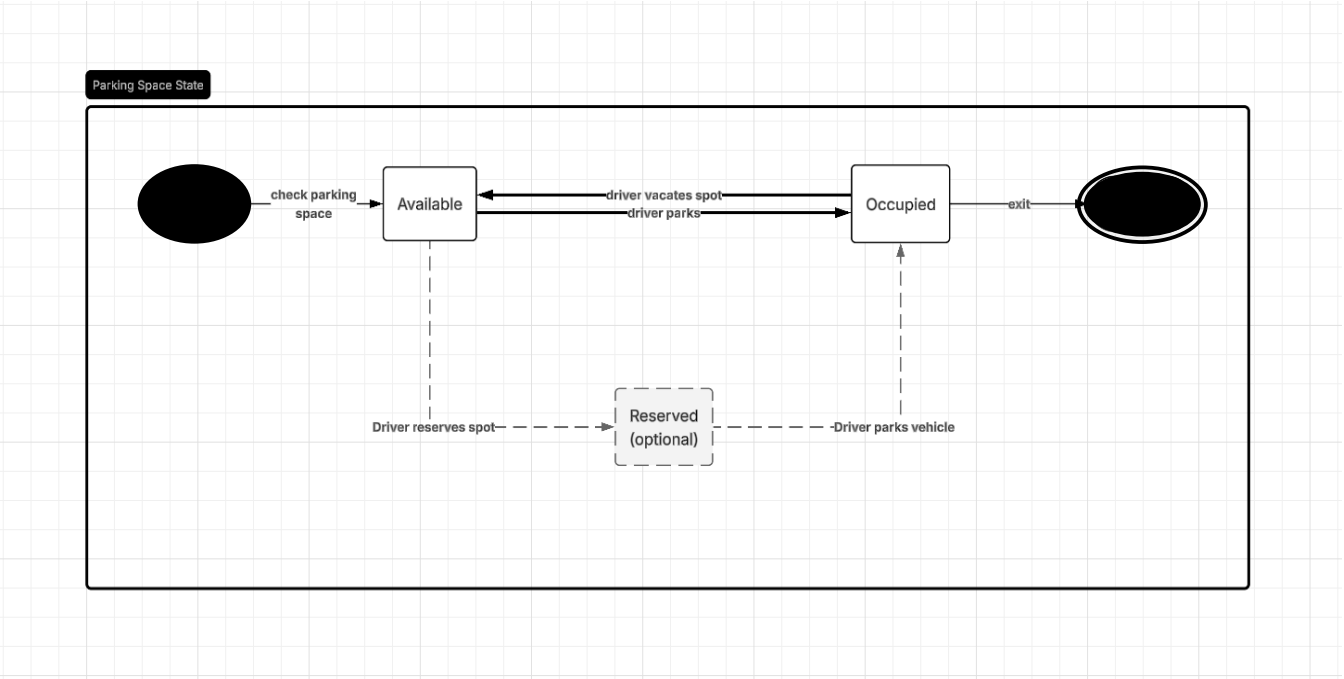
Overall, the diagram illustrates the interaction between users and the system, using «include» relationships for required functionalities and «extend» relationships for optional behaviors.

Activity Diagram

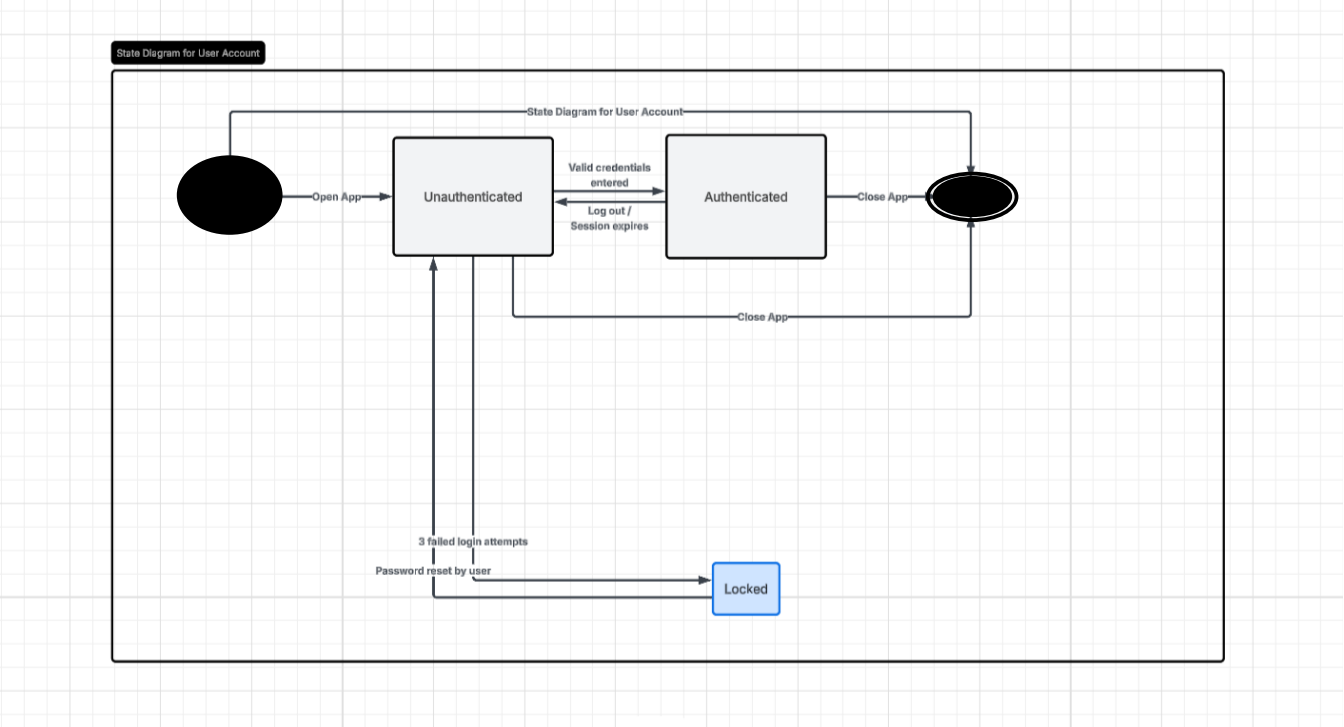


State Diagrams

1.Parking space state diagram



2.Driver account state diagram



Websites used for each diagram:

Component Diagrams: https://www.drawio.com/

Class Diagrams: https://www.lucidchart.com/pages

Sequence Diagrams: https://www.drawio.com/

Use Case Diagrams: https://www.drawio.com/

Activity Diagrams: https://www.drawio.com/

State diagram: https://www.lucidchart.com/pages